Lab Outcomes:

- Understand how to use comments in code to improve readability and disable code during testing.
- Learn how to handle user input for performing calculations or logic operations.
- Understand and practice basic program structure and syntax.
- Develop debugging skills to identify and resolve issues in the code.
- Use variables in mathematical and logical expressions to produce results based on user input.

C# Comments

Comments can be used to explain C# code, and to make it more readable. It can also be used to prevent execution when testing alternative code. Single-line comments start with two forward slashes (//).

```
// This is a comment

Console.WriteLine("Hello World!");
```

Multi-Line comment (/*comment*/).

```
/* The code below will print the words Hello World

to the screen, and it is amazing */

Console.WriteLine("Hello World!");
```

C# Variables

Variables are containers for storing data values.

In C#, there are different **types** of variables (defined with different keywords), for example:

- • int - stores integers (whole numbers), without decimals, such as 123 or -123

- • double - stores floating point numbers, with decimals, such as 19.99 or -19.99
- • char - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- • string - stores text, such as "Hello World". String values are surrounded by double quotes
- • bool - stores values with two states: true or false
- • long – stores large value for an integer

Constants

However, you can add the const keyword if you don't want others (or yourself) to overwrite existing values (this will declare the variable as "constant", which means unchangeable and readonly):

```
const int myNum = 15;
myNum = 20; // error
```

The const keyword is useful when you want a variable to always store the same value, so that others (or yourself) won't mess up your code. An example that is often referred to as a constant, is PI (3.14159...).

Note: You cannot declare a constant variable without assigning the value. If you do, an error will occur: A const field requires a value to be provided.

Declaration of Variables:

```
int myNum = 5;

double myDoubleNum = 5.99D;

char myLetter = 'D';

bool myBool = true;

string myText = "Hello";
```

Display Variables

```
string name = "John";

Console.WriteLine("Hello " + name);
```

You can also use the + character to add a variable to another variable:

```
string firstName = "John ";

string lastName = "Doe";

string fullName = firstName + lastName;

Console.WriteLine(fullName);
```

For numeric values the character works as a mathematical operator(notice the we use int(integer) variables here):

```
int x = 5;

int y = 6;

Console.WriteLine(x + y); // Print the value of x + y
```

C# Type Casting

Type casting is when you assign a value of one data type to another type.
In C#, there are two types of casting:

- **Implicit Casting** (automatically) - converting a smaller type to a larger type size
- char -> int -> long -> float -> double
- **Explicit Casting** (manually) - converting a larger type to a smaller size type
- double -> float -> long -> int -> char

Example:
Implicit casting int
myInt = 9;

```
double myDouble = myInt; // Automatic casting: int to double


Console.WriteLine(myInt); // Outputs 9
Console.WriteLine(myDouble); // Outputs 9


Explicit Casting double
myDouble = 9.78;
int myInt = (int) myDouble; // Manual casting: double to int  Console.WriteLine(myDouble);
// Outputs 9.78
Console.WriteLine(myInt); // Outputs 9
```

Type Conversion Methods

It is also possible to convert data types explicitly by using built-in methods, such as Convert.ToBoolean, Convert.ToDouble, Convert.ToString, Convert.ToInt32 (int) and Convert.ToInt64 (long):

Example:

```
int myInt = 10;  double
myDouble = 5.25;  bool
myBool = true;
Console.WriteLine(Convert.ToString(myInt)); // convert int to string
Console.WriteLine(Convert.ToDouble(myInt)); // convert int to double
Console.WriteLine(Convert.ToInt32(myDouble)); // convert double to int

Console.WriteLine(Convert.ToString(myBool)); // convert bool to string
```

Tasks:

Q1. Write a C# program where you declare variables of different data types, use single-line and multi-line comments to explain each section of the code, and define constants such as the value of PI to calculate the area and circumference of a circle using user input for radius.

Q2. Write a C# program that takes marks of five subjects from the user, uses a constant for total marks, performs explicit and implicit type casting to calculate the percentage, and then displays the result with appropriate type conversion methods.

Q3. Write a C# program that asks the user to enter their first and last name, age, and monthly income, stores them in appropriate variables, converts the income into yearly income using casting and conversion methods, combine first and last name as single name and displays all details in a properly formatted way with meaningful comments in the code.

Lab Assessment Criteria:
Lab #02 Marks distribution

	ER1	ER6	ER8
Task	3 points	3 points	4 points

Lab # 02
Rubric Evaluation Guideline:

#	**Qualities & Criteria**	**0 < Poor <=1**	**1 < Satisfactory <= 2**	**2 < Excellent <=3**
ER1	Task/Program Completion	Minimal or no task functionality was achieved	Some tasks were completed but few has errors or incomplete functionalities.	All tasks were completed without any errors.
#	**Qualities & Criteria**	**0 < Poor <=1**	**1 < Satisfactory <= 2**	**2 < Excellent <=3**
ER6	Code Structure and Design	Poor code structure and missing essential code elements like functions, data structures, or control flow.	Code functions but lacks proper structure, documentation, or uses inefficient algorithms.	Well-structured, documented code using appropriate data structures and algorithms.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1< Satisfactory <= 2.5**	**2.5< Excellent <= 4**
ER8	Question & Answer	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.